2026-06-10-tree-sitter-migration-plan

Tree-sitter Migration Plan: smacker/go-tree-sitter → tree-sitter/go-tree-sitter

Revision	1
Created	2026-06-10
Last modified	2026-06-10
Status	active
Maintainer	HelixCode CLI-Agent Fusion programme
Verdict	**DEFERRED-with-plan** — full swap is a genuine EPIC; both modules STAY on the working `smacker/go-tree-sitter` pin until the per-language gap + API-rewrite work is scheduled.

Table of contents

1. Verdict (TL;DR)

A full swap from `github.com/smacker/go-tree-sitter` (dormant since 2024-08-27, pinned at `v0.0.0-20240827094217-dd81d9e9be82` in BOTH modules) to the official `github.com/tree-sitter/go-tree-sitter` is **NOT a clean bounded swap**. It is a genuine epic for two independent reasons:

1. **Per-language module gap.** smacker bundles ~30 grammars as in-repo sub-packages (`smacker/go-tree-sitter/<lang>`). The official binding ships **zero** grammars — each language is a *separate* Go-module dependency at `github.com/<org>/tree-sitter-<lang>/bindings/go`. Of the 30 languages HelixCode/helix_agent use, **~6–8 have NO maintained official/first-party Go-module binding** (dockerfile, elm, protobuf, svelte, and others are personal forks or have no `bindings/go` at all). A full swap would force adding unvetted personal-fork Go modules — which collides with the project's catalogue-first / no-unvetted-dependency posture (§11.4.74) and the anti-bluff floor.

2. **API-shape rewrite at every call site.** The official lib is not a drop-in: package name, the grammar-loading idiom, the node accessor names (`Type()`→`Kind()`), the integer widths (`uint32`→`uint`), the `Parse` signature + return type (no `error` return), and the incremental-edit struct (`EditInput`→`InputEdit`, all fields renamed, value→pointer) all differ. Every one of the 4 production files + 1 test file must be rewritten, not re-imported.

Decision: leave both `go.mod` files on the smacker pin (it compiles and works today), do not force a half-migrated state (§11.4.101 — high-blast-radius change without the per-language groundwork). Schedule the epic per §6 when a maintainer owns the per-language vendoring + API-wrapper work.

The working tree was left **clean** — this assessment made zero source edits.

2. Scope: every smacker import site

Enumerated via `grep -rn "smacker/go-tree-sitter"` over `helix_code/` + `submodules/helix_agent/`.

Production code (4 files):

File	Languages imported	API used
`helix_code/internal/tools/mapping/treesitter_parsers.go`	26 langs: bash, c, cpp, csharp, css, dockerfile, elixir, elm, golang, hcl, html, java, javascript, lua, ocaml, php, protobuf, python, ruby, rust, scala, svelte, swift, toml, typescript (tsx + typescript), yaml	`sitter.NewParser/SetLanguage/ParseCtx`, `*sitter.Language`, `*sitter.Node` (`Type/StartByte/EndByte/StartPoint/EndPoint/ChildCount/Child/IsMissing`), `<lang>.GetLanguage()`
`helix_code/internal/repomap/tree_sitter.go`	9 langs: c, cpp, golang, java, javascript, python, ruby, rust, typescript	`sitter.NewParser`, `SetLanguage`, `ParseCtx`, `*sitter.Tree`, `Tree.Edit`, `Tree.Copy`, `sitter.EditInput`, `sitter.Point`
`helix_code/internal/repomap/tag_extractor.go`	(sitter root only)	`*sitter.Tree`, `*sitter.Node` walking
`submodules/helix_agent/internal/clis/aider/repo_map.go`	4 langs: golang, python, typescript/tsx, typescript	`*sitter.Parser`, `golang.GetLanguage()` etc.

Test code (1 file, also must migrate): - `helix_code/internal/repomap/incremental_p2t06_test.go` — exercises the incremental `EditInput` / `Copy` / `ParseContent` path; its assertions encode the smacker API and must be rewritten with the fix.

Doc-comment-only (no migration needed): `helix_code/internal/tools/mapping/doc.go` (lines 146–148) merely cites the import path in a `//` comment.

Both `go.mod` files pin `github.com/smacker/go-tree-sitter v0.0.0-20240827094217-dd81d9e9be82` (`helix_code/go.mod:42`, `submodules/helix_agent/go.mod:73`). Gin is already `v1.12.0` in both — no gin change is part of this epic.

3. API surface actually used + 1:1 mapping to the official lib

Concern	smacker (current)	tree-sitter/go-tree-sitter (target)
package	`sitter "github.com/smacker/go-tree-sitter"`	`tree_sitter "github.com/tree-sitter/ sitter"`
new parser	`sitter.NewParser()`	`tree_sitter.NewParser()` (+ `defer pars`
load grammar	`golang.GetLanguage()` → `*sitter.Language`	`tree_sitter.NewLanguage(tree_sitter_` (grammar `Language()` returns `unsafe.Pointe`
set language	`parser.SetLanguage(lang)`	`parser.SetLanguage(lang)`
parse	`parser.ParseCtx(ctx, oldTree, content) (*Tree, error)`	`parser.Parse(content, oldTree) *Tree (text, oldTree)`; deprecated `ParseCtx(ctx oldTree))`
node type	`node.Type() string`	`node.Kind() string`
byte offsets	`node.StartByte() uint32` / `EndByte() uint32`	`node.StartByte() uint` / `EndByte() uin`
points	`node.StartPoint()/EndPoint()` → `sitter.Point{Row,Column uint32}`	`node.StartPosition()/EndPosition()` — `tree_sitter.Point{Row,Column uint}`
children	`node.ChildCount() uint32`, `node.Child(i) *Node`	`node.ChildCount() uint`, `node.Child(i`
missing	`node.IsMissing() bool`	`node.IsMissing() bool`
edit struct	`sitter.EditInput{StartIndex, OldEndIndex, NewEndIndex, StartPoint, OldEndPoint, OldEndPoint}`	`tree_sitter.InputEdit{StartByte, Old NewEndByte, StartPosition, OldEndPos NewEndPosition}`
edit apply	`tree.Edit(editValue)`	`tree.Edit(&inputEdit)` (pointer)
tree copy	`tree.Copy() *Tree`	needs verification on target (`Tree.Copy` presence)
tree close	(smacker `tree.Close()`)	`tree.Close()`

The `repomap` package's incremental engine (`computeEditInput`, `IncrementalParser`, the `tree.Copy()`-per-retention design, `byteOffsetToPoint`) is the highest-touch surface: every `uint32` literal/cast and every `EditInput` field name changes, and the `Copy()` contract must be re-proven on the official lib before the incremental-vs-full equality tests can be re-greened.

4. Per-language module availability table (the blocker)

For each of the 30 used grammars, the official-ecosystem Go-module path and maintenance status. "✅" = a maintained module with `bindings/go` exists under an official-ish org; "⚠️" = community-fork only / no first-party Go binding → would add an unvetted dependency.

Language	Target Go module (if any)	Status
go (golang)	`tree-sitter/tree-sitter-go/bindings/go`	✅ official
javascript	`tree-sitter/tree-sitter-javascript/bindings/go`	✅ official
typescript + tsx	`tree-sitter/tree-sitter-typescript/bindings/go`	✅ official
python	`tree-sitter/tree-sitter-python/bindings/go`	✅ official
rust	`tree-sitter/tree-sitter-rust/bindings/go`	✅ official
java	`tree-sitter/tree-sitter-java/bindings/go`	✅ official
c	`tree-sitter/tree-sitter-c/bindings/go`	✅ official
cpp	`tree-sitter/tree-sitter-cpp/bindings/go`	✅ official
csharp	`tree-sitter/tree-sitter-c-sharp/bindings/go`	✅ official
ruby	`tree-sitter/tree-sitter-ruby/bindings/go`	✅ official
php	`tree-sitter/tree-sitter-php/bindings/go`	✅ official
scala		✅ official

Language	Target Go module (if any)	Status
	`tree-sitter/tree-sitter-scala/bindings/go`	
ocaml	`tree-sitter/tree-sitter-ocaml/bindings/go`	✅ official
html	`tree-sitter/tree-sitter-html/bindings/go`	✅ official
css	`tree-sitter/tree-sitter-css/bindings/go`	✅ official
bash	`tree-sitter/tree-sitter-bash/bindings/go`	✅ official
toml	`tree-sitter-grammars/tree-sitter-toml/bindings/go`	✅ community-official org
hcl	`tree-sitter-grammars/tree-sitter-hcl/bindings/go`	✅ community-official org
lua	`tree-sitter-grammars/tree-sitter-lua/bindings/go`	✅ community-official org (verify binding)
yaml	`tree-sitter-grammars/tree-sitter-yaml/bindings/go`	⚠️ verify `bindings/go` present
swift	`alex-pinkus/tree-sitter-swift`	⚠️ generated-parser repo, no clean `bindings/go` go-get
elixir	`elixir-lang/tree-sitter-elixir`	⚠️ verify `bindings/go`; not in `tree-sitter/*`
elm	`elm-tooling/tree-sitter-elm`	⚠️ no first-party Go binding
dockerfile	`camdencheek/tree-sitter-dockerfile`	⚠️ personal repo, no official Go binding
protobuf	only personal forks (`dgmcdona/...`, `zzctmac/...`, etc.)	⚠️ unvetted fork only
svelte	only personal forks (`kyouheicf/...`)	⚠️ unvetted fork only

~6–8 of 30 fall in the ⚠ band. A full swap cannot ship without either (a) accepting unvetted third-party Go modules for those grammars (anti-bluff / §11.4.74 concern), or (b) vendoring the generated C parser + writing a thin local `bindings/go` shim per missing grammar (real engineering per language).

5. Why this is an EPIC, not a bounded swap

- **Not all languages available** → cannot do a one-shot `go.mod` line swap; the missing grammars each need a vendoring/shim decision.
- **API is not 1:1** → `Type()`→`Kind()`, `uint32`→`uint` everywhere, `ParseCtx(ctx,old,text)`→`Parse(text,old)` with no error return, `EditInput`→`InputEdit` (renamed struct + fields + value→pointer). Every call site is a rewrite, and the `repomap` incremental engine + its equality tests are high-risk.
- **Two modules, recursively** → `helix_code` (35 import lines across 3 prod files + 1 test) and `helix_agent` (4 import lines, 1 prod file) must both migrate and both keep building.
- **Anti-bluff cost** → per §11.4.43/§11.4.115 the incremental-parse equality + symbol-extraction tests must go RED on the new API, then GREEN — the migration is only "done" with captured `go build ./...` exit-0 + the repomap/mapping unit tests green on the new lib, not a compile-only check.

6. Per-module migration steps (when scheduled)

Recommended order: a thin internal adapter first, so the 30-language fan-out and the API-shape rewrite are isolated from each other.

1. **Adapter package** (`internal/repomap/tsadapter` or similar): define HelixCode-local `Language`, `Parser`, `Node`, `Tree`, `EditInput` types that wrap the official lib and preserve the *current* call-site signatures (`Type()`, `uint32`, `ParseCtx`-style, value-`EditInput`). This localises the `Kind()`/`uint`/`Parse`/`InputEdit` translation to ONE file and keeps the 4 call-site files small diffs. Verify `Tree.Copy` exists on the target; if not, implement copy semantics in the adapter.
2. **Grammar registry**: a single `GetLanguage(name)` in the adapter that maps name→official grammar module. For each ⚠ language (dockerfile, elm, protobuf, svelte, swift, elixir, yaml): decide vendor-generated-parser vs accept-fork vs drop-language-with-operator-approval (§11.4.122 — dropping a supported language is an operator-confirmed removal, never silent).
3. **helix_code first**: bump `helix_code/go.mod` to add `tree-sitter/go-tree-sitter` + each available grammar module; rewrite the 4 files through the adapter; `go mod tidy`; `go build ./...` exit 0; `go test ./internal/repomap/... ./internal/tools/mapping/...` green (incl. the rewritten `incremental_p2t06_test.go`). RED→GREEN on the incremental-equality assertions.
4. **helix_agent second**: same adapter pattern (or a copy of the adapter, since modules are decoupled per §11.4.28 — do NOT cross-import); rewrite `repo_map.go`; build + test green.
5. **Remove smacker pin** from both `go.mod` only after both build+test green; `go mod tidy`.
6. **Capture evidence**: pasted `go build ./...` + `go test` output per module per §11.4.5 / Rule 8.

7. Risk register

Risk	Severity	Mitigation
⚠️ grammars force unvetted fork deps	High	adapter step 2 decision per language; operator-confirm any language drop (§11.4.122)
`uint32`→`uint` width change breaks `int(...)` casts silently	Medium	adapter normalises to the current types; build catches the rest
incremental `Copy()` contract differs/absent on official lib	Medium	verify before relying; adapter shim if needed; equality tests are the proof
`cgo`/build-tag differences across the per-language modules	Medium	`go build ./...` on each target platform in the matrix
two modules drift if migrated unevenly	Low	migrate+build+test helix_code fully before helix_agent

8. Sources verified

Sources verified 2026-06-10: - https://github.com/tree-sitter/go-tree-sitter — official binding, “none of the grammars are included”, per-language `go get`. - https://pkg.go.dev/github.com/tree-sitter/go-tree-sitter — `Parse(text, oldTree)`, `Node.Kind()`, `uint` widths, `InputEdit{StartByte,...}`, `Point{Row,Column uint}`, `Tree.Edit(*InputEdit)`. - https://github.com/tree-sitter-grammars — `tree-sitter-grammars/*` org hosts toml/hcl/lua/etc. `bindings/go`. - https://github.com/camdencheek/tree-sitter-dockerfile, https://github.com/elm-tooling/tree-sitter-elm — dockerfile/elm not under official org / no first-party Go binding. - https://pkg.go.dev/github.com/smacker/go-tree-sitter — current binding being replaced.